

qis: reproducible performance analytics and portfolio backtesting in Python

Artur Sepp¹

1 LGT Bank (Schweiz) AG, Zürich, Switzerland

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 29 July 2026

Published: unpublished

License

Authors of papers retain copyright, and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

qis computes the quantities a quantitative portfolio manager or investor reports. It turns a series of portfolio weights, generated externally, and a panel of underlying asset prices into a simulated portfolio, measures it against a benchmark, and renders the result as a factsheet. In addition to the backtesting engine, qis provides strategy and reporting building blocks: exponentially weighted covariance, block bootstrap resampling, risk measures, regime-conditional statistics, currency hedging, and return unsmoothing.

We designed qis around one assumption: a strategy is a rule for producing weights, and, for the simulation and reporting workflows qis targets, everything downstream of those weights is reconstruction rather than research. The researcher develops the signal and the portfolio construction — the things that actually matter for a successful investment process; the rest belongs to the library.

qis is the base layer of a stack of quantitative finance packages — optimalportfolios, trendfollowing and privateassets, all three public. Each capability is written once and reused across that stack. It requires only the scientific Python stack, runs on Python 3.10 to 3.14, and is released under the MIT licence.

Statement of need

Reported portfolio performance numbers depend on conventions that are rarely stated. Volatility depends on the return frequency, a Sharpe ratio on whether a risk-free rate is subtracted and on which rate, and a backtested return on whether holdings drift between rebalancings or are reset every period. Two implementations of the same strategy can therefore report different numbers without either containing an error.

We answer each of these questions once, inside the library, and report the answer beside the number. The return convention is an argument rather than an assumption, three Sharpe conventions are named and selected explicitly, and the reporting frequency appears on every rendered panel. The simulation holds units between rebalancings, so the realised weights drift with prices, which is what a portfolio does.

The cost of leaving a convention unstated is measurable rather than notional. The stationary bootstrap resamples a series in blocks, and a block drawn near the end of the sample either wraps around to the start or stops there. Both forms run, and neither reports which it used. Under the truncating form the first observation of a 250-period sample is drawn at 0.11 times its uniform weight and the first decile at 0.53 times, because a block can only run forwards. On a series whose drift rises through the sample, that uneven draw reports a mean return 2.15% per year above the source, while the wrapping form reports it within 0.32%. Two researchers running a stationary bootstrap on the same data would publish annual returns more than two

percentage points apart, and nothing in either output would explain the gap. The example producing these numbers ships with the package, and its values are pinned by the test suite.

A backtest needs two inputs: a strategy generator produces weights, and a price panel aligned to those weights supplies everything else. When each consumer of a result rebuilds the reported quantities for itself, the Sharpe ratio of one strategy, read from a chart, a summary table and a factsheet, comes out as three numbers. We build all three from one `PortfolioData` object constructed from weights and prices, and from the same object come the comparisons a pipeline actually runs. In addition, we implement two other base pipelines for strategy profiling: a bumped strategy against its base, and several building blocks of the same strategy side by side. All three reports are rendered by one reporting layer rather than a figure written per comparison.

The same discipline answers a second need. A research group accumulates analytics faster than it consolidates them, and the same method is written again in each project. In our own repositories we found four independent block bootstrap implementations and two return unsmoothers, all reimplementing code `qis` already exported. They had diverged, so the same nominal method produced different numbers in different papers.

The argument extends to agentic AI tools: a researcher who directs one at `qis` for simulation and reporting, rather than letting it write those steps, holds the conventions fixed across sessions instead of having each session reimplement them and reproduce the divergence above.

State of the field

Packages in this area are usually described as splitting into reporting libraries and backtesting frameworks; their documented interfaces do not, so the comparison below is capability by capability, at the versions current on 27 July 2026.

package	its documented interface takes	and returns
pyfolio-reloaded 0.9.9	returns, and optionally positions, transactions, market data, factor returns and loadings	tear sheets, including round-trip analysis rebuilt from positions and transactions
bt 1.2.0	<code>WeighTarget</code> takes a target-weight <code>DataFrame</code> from the caller, as part of an Algo-stack strategy	a result exposing statistics, weights, positions and turnover
vectorbt 1.1.0	<code>Portfolio.from_orders</code> , <code>from_signals</code> and <code>from_order_func</code> take orders, signals, or an order-generating callback produced elsewhere	order and trade records, statistics and plots
Riskfolio-Lib 7.3.0	returns and optimisation constraints	weights, with documented <code>Reports</code> and <code>PlotFunctions</code> modules
skfolio 0.20.1	a scikit-learn estimator fitted to returns	weights, walk-forward and combinatorial purged cross-validation, risk measures and plots
quantstats 0.0.81	a return series, not trade data	metrics, plots and an HTML tear sheet

package	its documented interface takes	and returns
empyrial-reloaded 0.5.12	a return series	return and risk metrics as functions, with no reporting layer
ffn 1.1.5	prices	analytics; its documentation refers simulation to bt
PyPortfolioOpt 1.6.0	prices, or expected returns and a risk model	weights and a discrete allocation, <code>portfolio_performance()</code> , a plotting module

63 arch has no row, being a resampling and econometrics library rather than a portfolio stack,
64 but it documents the closest primitive: `StationaryBootstrap` takes several aligned arrays,
65 `Series` or `DataFrames` in one object, so one index draw propagates across them.

66 `qis` differs in where the boundary sits. Only weights and prices cross it, so any generator
67 producing weights can be measured. The simulation returns a `PortfolioData` object carrying
68 the net asset value, the realised weights, the held units, the instrument-level profit and loss
69 and the realised costs together, so attribution is a property of that object rather than a later
70 calculation on a series that has already lost what it needs.

71 Two capabilities have no counterpart in the documented interfaces above. Return unsmoothing
72 corrects the serial correlation induced by appraisal-based valuation, which matters for private
73 assets and hedge funds (Getmansky et al., 2004). Regime-conditional reporting partitions
74 every statistic by benchmark return quantile (Sepp & Dezaud, 2019). A third, paired block
75 bootstrap resampling — one index draw across several aligned panels, so a factor and a residual
76 panel resample together (Politis & Romano, 1994) — has the primitive noted above.

77 Software design

78 We organise the package into 14 capability groups, which `src/qis/api.py` names.

79 The backtester is 427 lines, and that follows from the design assumption rather than from
80 compression. Because a strategy is a rule for producing weights, the backtester holds no
81 strategy logic: it converts target weights to units at each rebalancing, holds those units until
82 the next one, applies costs, and returns the result. The recursion over dates is compiled with
83 `numba`, one of the few loops here that cannot be vectorised.

84 The public interface is `qis.__all__`, which holds 464 names; `src/qis/api.py` records that
85 list as a literal together with a documented core of 158 symbols grouped by capability, and the
86 suite fails when either record disagrees with the namespace.

87 Properties are enforced by tests rather than by convention: every exported plotting function
88 draws a figure on a synthetic panel; every example references symbols and keyword arguments
89 that exist; every core symbol documents its arguments and documents none it does not take;
90 documentation links resolve; and the measurements this paper quotes match a generated
91 record. Every example that needs no data vendor runs in the suite; the rest are checked
92 statically.

93 The suite runs without network access on a core installation, from a frozen seeded simulator
94 reproducing the defects of real panels: ragged starts, missing observations, stale prices, appraisal
95 smoothing.

Research impact statement

We state plainly that the stack is the author's. Two public consumers carry this section. `optimalportfolios` declares `qis` a mandatory dependency and references 101 of its symbols at 832 sites. `trendfollowing`, which carries the trend-following work cited below, references 87 symbols at 441 sites. Both counts are taken at the commits recorded in `docs/audit/consumers.json` and reproduced by `tools/audit_consumers.py --pinned`.

A third public package, `privateassets`, applies the unsmoothing layer to private-asset returns; we cite it for the range of asset classes served, not as adoption evidence.

The capabilities carry named results. The portfolio and optimisation layers support work on robust strategic and tactical allocation (Sepp, Ossa, et al., 2026), the backtester and performance layer work on cryptocurrency allocation (Sepp, 2023), the regime-conditional layer work on trend-following systems (Sepp & Lucic, 2026), and the resampling layer work on capital market assumptions built from multi-asset tradable factors (Sepp, Hansen, et al., 2026). The last two are working papers.

Commits run from December 2022 to September 2026, with activity in 39 of the 46 calendar months.

AI usage disclosure

The analytical methods, the conventions they implement, and the architecture of the package are the author's, and no method in `qis` is originated from a language model.

In July 2026 the author used Anthropic's Claude (2026 models), through an agentic coding interface, for a documentation and test infrastructure effort preceding this submission. That work produced the test suite described above, the docstrings on the documented core, the documentation configuration, the audit scripts under `tools/`, and repairs of defects the new tests exposed. It also implemented two changes the author specified: circular block wrapping in the stationary bootstrap, and a fixed-coefficient option in the static unsmoothing estimator. The co-authored commits can be listed with `git log --grep='Co-authored-by: Claude'`; their number is not quoted here because it moves.

We drafted this paper with the same assistance, working from measurements the assistant recorded under `docs/audit/`. Every row of the comparison above, and every bibliographic claim, was verified against primary documentation on 29 July 2026. The author reviewed, edited and validated all assisted output, wrote the economic argument in the statement of need, made the design decisions, and is responsible for the content.

References

Getmansky, M., Lo, A. W., & Makarov, I. (2004). An econometric model of serial correlation and illiquidity in hedge fund returns. *Journal of Financial Economics*, 74(3), 529–609. <https://doi.org/10.1016/j.jfineco.2004.04.001>

Politis, D. N., & Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303–1313. <https://doi.org/10.1080/01621459.1994.10476870>

Sepp, A. (2023). Optimal allocation to cryptocurrencies in diversified portfolios. *Risk Magazine*, 1–6. <https://doi.org/10.2139/ssrn.4217841>

Sepp, A., & Dezaud, L. (2019). Trend-following CTAs vs alternative risk-premia: Crisis beta vs risk-premia alpha. *The Hedge Fund Journal*, 138, 20–31. <https://thehedgefundjournal.com/trend-following-ctas-vs-alternative-risk-premia/>

- 140 Sepp, A., Hansen, E. H., & Kastenholz, M. (2026). *Capital market assumptions and strategic*
141 *asset allocation using multi-asset tradable factors* [Working paper]. [https://doi.org/10.](https://doi.org/10.2139/ssrn.6785958)
142 [2139/ssrn.6785958](https://doi.org/10.2139/ssrn.6785958)
- 143 Sepp, A., & Lucic, V. (2026). *The science and practice of trend-following systems* [Working
144 paper]. <https://doi.org/10.48550/arXiv.2607.19497>
- 145 Sepp, A., Ossa, I., & Kastenholz, M. (2026). Robust optimization of strategic and tactical
146 asset allocation for multi-asset portfolios. *The Journal of Portfolio Management*, 52(4),
147 86–120. <https://doi.org/10.3905/jpm.2025.1.806>

DRAFT